Project Report – Hospital appointment and Patient Management System

Submitted by:
A Aaryan Dharrmik (Reg. No: 2460301, a.aaryan@btech.christuniversity.in)
Tirth Patel (Reg. No: 2460432, tirth.manishbhai@btech.christuniversity.in)
Amiya Prakash (Reg. No: 2460325, amiya.prakash@btech.christuniversity.in)
Sneha Rohith (Reg. No: 2460453, sneha.rohith@btech.christuniversity.in)

Course: Advanced Javascript Framework (Angular)
Institution: CHRIST University Kengeri
Date of Submission: 10-03-2026

Github link :
https://github.com/Aaryan658/L-T-Hospital-appointment-and-Patient-Management-System

Objective

The primary objective of this project is to design and develop a robust application for managing hospital appointments and patient records. Utilizing Angular 17 and its modern Standalone Component architecture, the system aims to simulate a digital healthcare environment.
The project focuses on creating distinct user experiences for doctors and patients, enabling functionalities such as appointment booking, status tracking, and medical history review. A key goal is to implement strict separation of concerns through clean code practices, domain modeling, and mock data simulation without relying on an external backend in this initial phase.

Technologies Used

Angular 17: Core framework utilizing the latest features like Standalone Components, Control Flow syntax (@if, @for), and Signals.
TypeScript 5.2: For strong typing, interfaces, and object-oriented structure.
HTML5 & SCSS: For semantic structure and modular, component-specific styling.
Angular CLI: For scaffolding, building, and serving the application.
Google Fonts: Playfair Display (Headers) and Inter (Body) for typography.

Features

1. Role-Based Access Control (Mock): Distinct login flows for Patients (via Patient ID) and Doctors (via Doctor ID) using simulated authentication guards.
2. Patient Dashboard: A dedicated view for patients to check upcoming appointments, view past medical history, and book new consultations.
3. Doctor Dashboard: A management interface allowing doctors to accept or decline appointment requests and view their daily schedule.
4. Appointment Booking System: An intuitive wizard allowing patients to filter doctors by specialization and select available time slots.
5. Mock Data Persistence: Utilization of TypeScript services to hold in-memory data states, ensuring data consistency during the user session.
6. Responsive Interface: A layout that adapts seamlessly across desktops, tablets, and mobile devices using CSS Grid and Flexbox.

Module Description

1. **Authentication Module**
   Handles the entry point of the application. It includes the which accepts a User ID. Based on the ID prefix (e.g., ' PT' for Patient, ' DR' for Doctor), the system routes the
   user to the appropriate dashboard. It uses a to simulate session management.

2. **Patient Portal**
   Dashboard Component: Displays a summary of "Next Appointment" and "Recent Activity". Booking Component: Contains a form with dropdowns for Specialization and Doctor selection. Uses Angular' s Reactive Forms for validation.
   History Component: A tabular view of past consultations.

3. **Doctor Portal**

Appointment Manager: Lists pending appointment requests. Doctors can click "Approve" or "Reject", which updates the status in the centralized Mock Data Service.
Schedule View: A calendar-like list view of confirmed appointments for the day.

4. **Shared Components**
   Header/Navigation: Context-aware navigation bar that changes links based on the logged-in user role.
   Status Badge: A reusable presentation component to display appointment status (Pending, Confirmed, Cancelled) with color-coding.

Architecture & Design

Angular Standalone Architecture
The application moves away from the traditional NgModule pattern. All components, directives, and pipes are marked as standalone: true . This reduces boilerplate and improves tree-shaking performance.

Component Hierarchy

```
src/app/
├── components/          # Presentational Components
│   ├── appointment-form
│   ├── doctor-dashboard
│   ├── doctor-list
│   ├── navbar
│   └── patient-dashboard
├── mock-data/           # Static data simulation (API Abstraction)
├── models/              # Pure TypeScript Interfaces
├── app.component.ts     # State Container & Logic
└── app.html             # Main View Template
```

Domain Models

Strict TypeScript interfaces define the shape of data, ensuring type safety throughout the application.
Appointment:{ id, patientId, doctorId, preferredDate, status, type, reason }
Doctor: { id, name, specialization, department, status, experience, qualification }
Patient: { id, name, email, contactNumber, age, gender, bloodGroup }
User: (Refers to the authenticated entity) { id, role (patient | doctor) }

Future Enhancements

- Backend Integration: Connect to a Node.js/Express API with a MongoDB database to replace mock data.
- Real-time Notifications: Implement WebSockets (Socket.io) to alert doctors immediately when a new appointment is booked.
- Telemedicine Integration: Embed WebRTC for video consultations directly within the browser.
- JWT Authentication: Replace simple ID checks with secure JSON Web Token authentication and route guards.
- Pharmacy Module: Add e-prescription capabilities linked to a pharmacy inventory system.

Outcome

The project successfully delivers a functional Phase-1 prototype of the Hospital Appointment System. The application validates the architectural choice of Standalone Components, demonstrating faster load times and simplified routing configuration. The UI allows for complete user flows—from login to booking and approval—validating the business logic before backend integration.
The strict separation of presentation (Components) and logic (Services) ensures that the codebase is maintainable and ready for future scaling.

Challenges Faced and Solutions

Challenge: State Persistence without Backend
Problem: Ensuring that appointments booked by a patient appeared instantly on the doctor's dashboard without a database.

Solution: Implemented a singleton DataService using RxJS BehaviorSubject. This acts as an in-memory store, holding the current state of appointments and emitting updates to all subscribers instantly.

Challenge: Complex Filtering Logic Problem: Filtering doctors based on multiple criteria (specialization, availability) in the booking wizard caused code complexity.

Solution: Utilized Angular Pipes (DoctorFilterPipe) to completely encapsulate the filtering logic. This allows for clean template binding and automatic processing of the doctor list based on Search, Specialization, and Status inputs without bloating the component code.

Sample Code

1. Data Model (Appointment)
Demonstrates TypeScript interfaces and strong typing for domain entities.

File: src/app/models/appointment.model.ts

```
export interface Appointment {
  /** Unique appointment identifier */
  id: number;

  /** FLKey: Reference to the Patient entity */
  patientId: number;

  /** FLKey: Reference to the Doctor entity */
  doctorId: number;

  /** Scheduled date and time (ISO string) - Preferred Date for request */
  preferredDate: string;

  /** Current state of the appointment workflow */
  status: 'Requested' | 'Pending' | 'Approved' | 'Rejected' | 'Cancelled';

  /** Type of visit */
  type: 'Initial Consultation' | 'Follow-up' | 'Routine Checkup' | 'Emergency';

  /** Optional notes or reason provided by the patient */
  reason?: string;

  /** Timestamp of record creation */
  createdAt: string;
}
```

2. Service Layer (AppointmentService)
Demonstrates Dependency Injection, RxJS Observables, and in-memory state management.

File: src/app/services/appointment.service.ts

```
@Injectable({
  providedIn: 'root'
})
export class AppointmentService {
  // In-memory state management for session persistence
  private appointments: Appointment[] = [...APPOINTMENTS];
  private appointmentsSubject = new
BehaviorSubject<Appointment[]>(this.appointments);

  constructor(private http: HttpClient) { }

  getAppointments(): Observable<Appointment[]> {
    return this.appointmentsSubject.asObservable();
  }

  createAppointment(appointment: Appointment): Observable<Appointment> {
    // Simulate backend ID generation
    appointment.id = this.appointments.length > 0 ?
Math.max(...this.appointments.map(a => a.id)) + 1 : 101;
    appointment.createdAt = new Date().toISOString();
    appointment.status = 'Requested';

    this.appointments = [...this.appointments, appointment];
    this.appointmentsSubject.next(this.appointments);

    return of(appointment).pipe(delay(500));
  }

  updateAppointmentStatus(id: number, status: 'Approved' | 'Rejected' | 'Cancelled'):
Observable<Appointment | undefined> {
    const index = this.appointments.findIndex(a => a.id === id);
    if (index !== -1) {
      const updatedAppointment = { ...this.appointments[index], status };
      this.appointments[index] = updatedAppointment;
      this.appointments = [...this.appointments]; // Immutable update
      this.appointmentsSubject.next(this.appointments);
      return of(updatedAppointment).pipe(delay(300));
    }
    return of(undefined);
```

```
    }
}
```

3. Security Guard (AuthGuard)
Demonstrates route protection and role-based access control (RBAC).

File:
src/app/guards/auth.guard.ts

```
export const authGuard: CanActivateFn = (route, state) => {
    const router = inject(Router);
    // Simulation: Check localStorage or a mock auth state
    const userRole = localStorage.getItem('userRole'); // 'patient' | 'doctor' | null

    const expectedRole = route.data['role'];

    if (!userRole) {
        return router.createUrlTree(['/login']);
    }

    if (expectedRole && userRole !== expectedRole) {
        // Redirect to appropriate dashboard if role mismatch
        if (userRole === 'patient') return router.createUrlTree(['/patient-dashboard']);
        if (userRole === 'doctor') return router.createUrlTree(['/doctor-dashboard']);
        return false;
    }

    return true;
};
```

4. Component Logic (DoctorDashboardComponent)
Demonstrates Component-Based Architecture, data binding, and filtering logic.

File: src/app/components/doctor-dashboard/doctor-dashboard.component.ts

```
@Component({
    selector: 'app-doctor-dashboard',
    standalone: true,
    imports: [CommonModule, MatCardModule, /*...*/],
    templateUrl: './doctor-dashboard.component.html',
```

```
    styleUrl: './doctor-dashboard.component.css'
})
export class DoctorDashboardComponent implements OnInit {
    appointments: Appointment[] = [];
    // ...

    constructor(private appointmentService: AppointmentService, /*...*/) { }

    ngOnInit() {
        // ...
        this.appointmentService.getAppointments().subscribe(data => this.appointments =
data);
        // ...
    }

    get pendingAppointments() {
        return this.appointments.filter(a => a.status === 'Requested');
    }

    approveRequest(appointment: Appointment) {
        this.appointmentService.updateAppointmentStatus(appointment.id,
'Approved').subscribe(() => {
            // Local optimistic update
            const index = this.appointments.findIndex(a => a.id === appointment.id);
            if (index !== -1) {
                this.appointments[index].status = 'Approved';
            }
        });
    }
}
```

5. UI Template (Doctor Dashboard)
Demonstrates Angular Material usage, structural directives (*ngIf, *ngFor), and Pipes.

File:
src/app/components/doctor-dashboard/doctor-dashboard.component.html

```
<mat-tab-group animationDuration="0ms">
    <mat-tab label="Appointment Requests">
        <div class="card-grid" *ngIf="pendingAppointments.length > 0; else noRequests">
```

```
        <mat-card *ngFor="let appointment of pendingAppointments"
class="appointment-card pending">
            <mat-card-header>
                <div mat-card-avatar class="patient-avatar-sm">
                    <mat-icon>person</mat-icon>
                </div>
                <mat-card-title>{{ getPatientName(appointment.patientId)
}}</mat-card-title>
                <mat-card-subtitle *ngIf="getPatient(appointment.patientId) as p">
                     <!-- Patient Identity Info -->
                    <div class="patient-quick-info">
                        <span class="patient-id">ID: P00{{ p.id }}</span>
                        <span class="info-text">{{ p.age }} Years • {{ p.gender }} • {{
p.bloodGroup }}</span>
                    </div>
                </mat-card-subtitle>
            </mat-card-header>
            <mat-card-actions align="end" class="action-buttons">
                <button mat-button class="reject-btn"
(click)="rejectRequest(appointment)">Reject</button>
                <button mat-raised-button color="primary"
(click)="approveRequest(appointment)">Approve</button>
            </mat-card-actions>
        </mat-card>
    </div>
    <ng-template #noRequests>
        <div class="empty-state">
            <mat-icon>inbox</mat-icon>
            <h3>No Pending Requests</h3>
        </div>
    </ng-template>
  </mat-tab>
</mat-tab-group>
```

Results

The following screenshots demonstrate the core functionalities of the application.

Figure 1: Authentication Screen

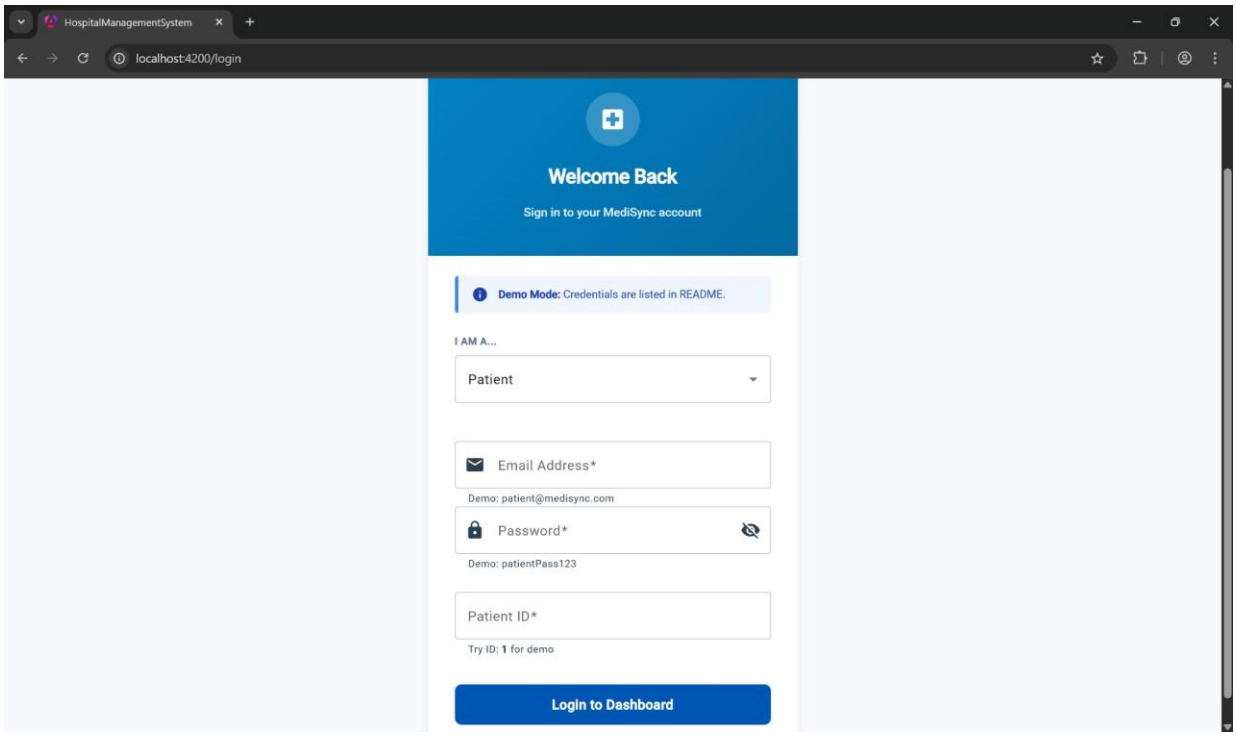


Figure 2: Patient Dashboard viewing available doctors

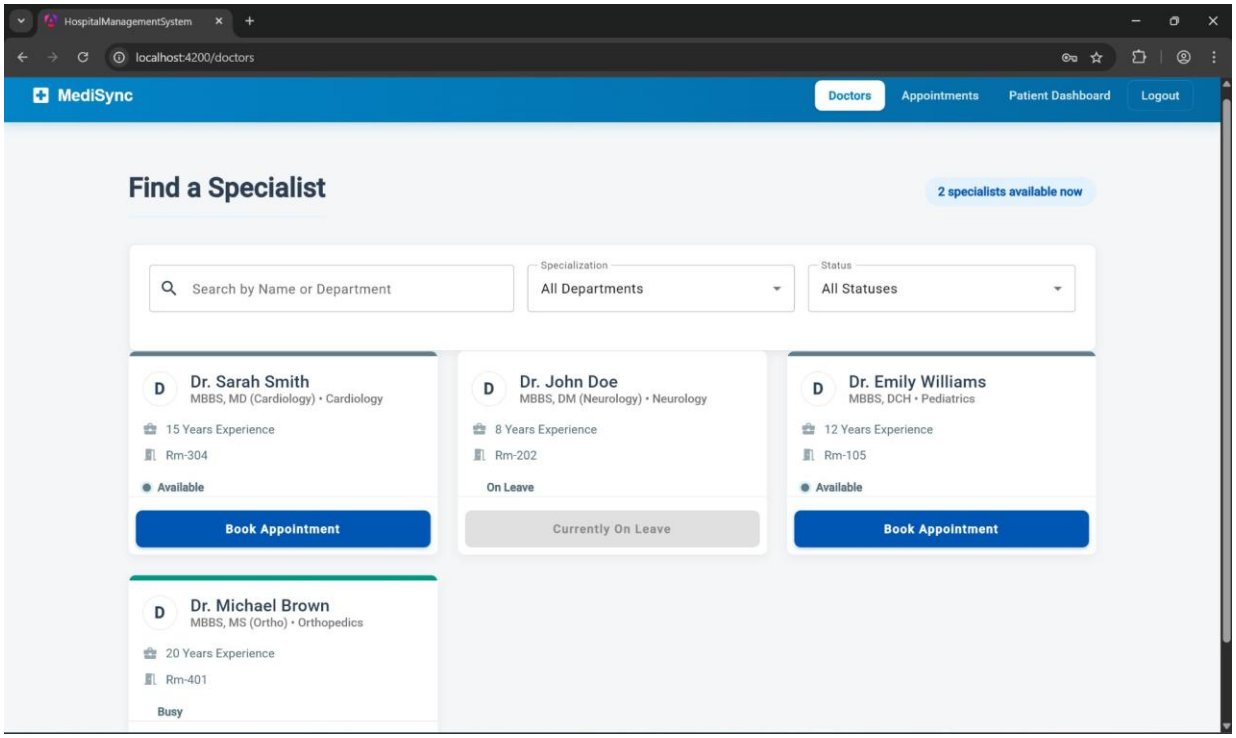


Figure 3: Patient Dashboard showing how to book an appointment

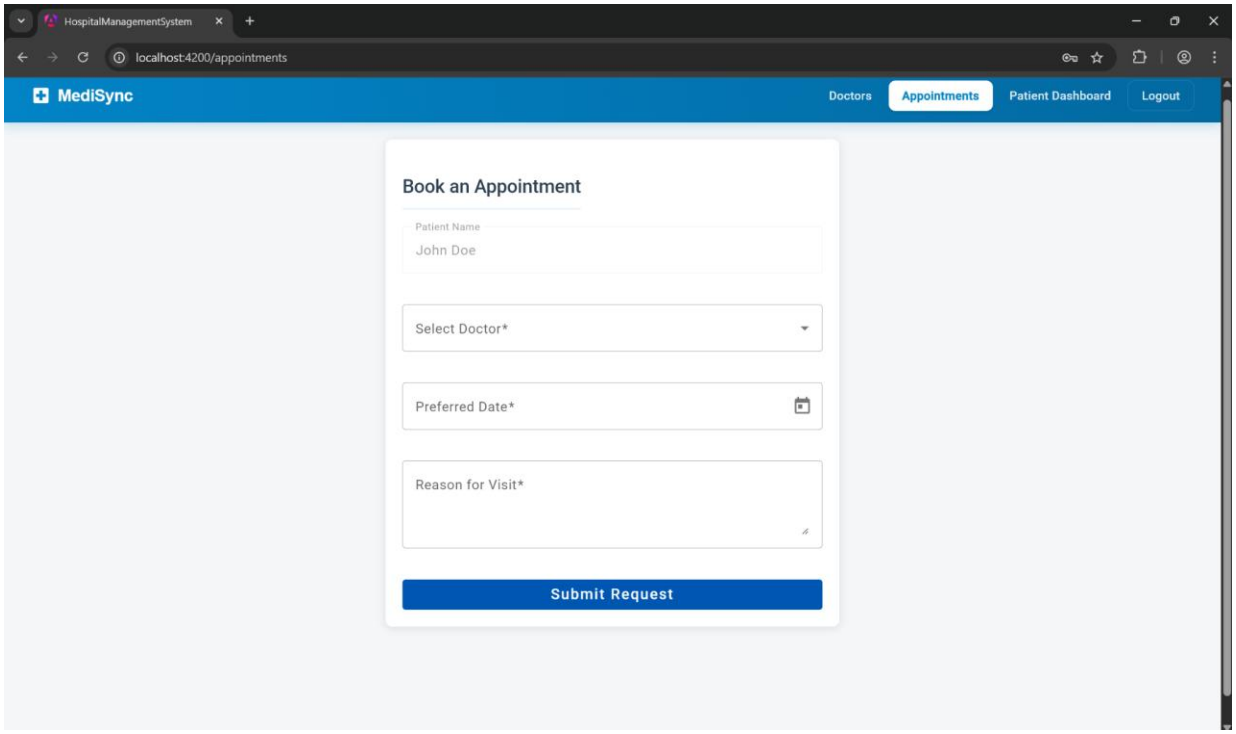


Figure 4: Patient Dashboard showing patient details

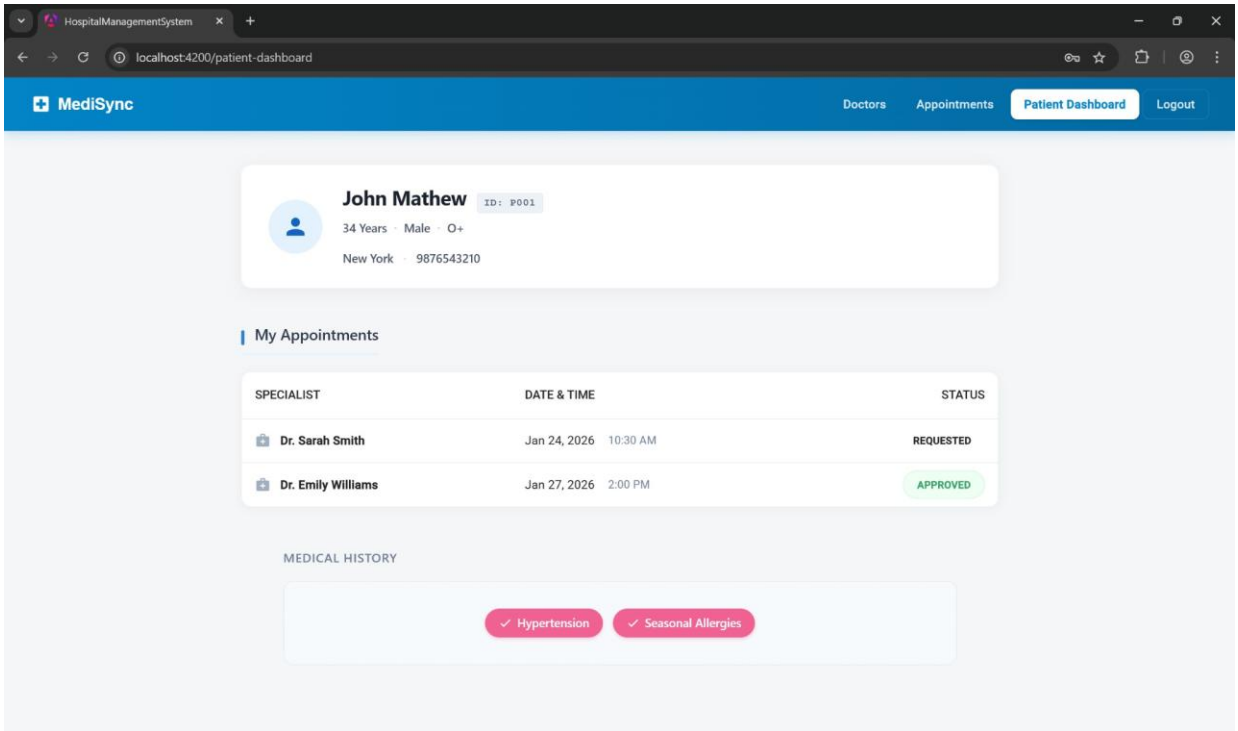
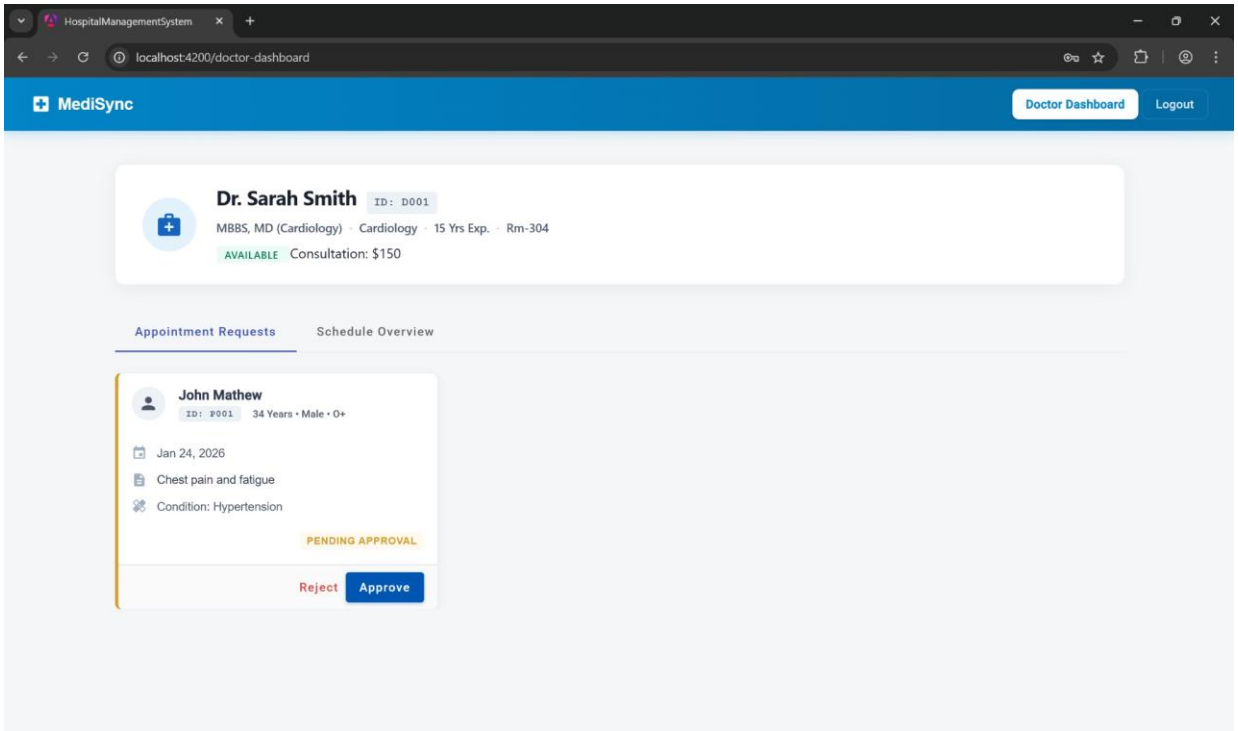


Figure 5: Doctor Dashboard managing pending approvals



Conclusion

This project successfully demonstrates the power and flexibility of Angular 17. By adhering to the Standalone Component architecture and utilizing modern TypeScript features, the application achieves a high degree of modularity and maintainability.

Although currently operating on mock data, the system architecture is robust enough to support seamless integration with a real backend in future phases. The project fulfills its objective of simulating a comprehensive digital healthcare workflow, providing valuable insights into frontend architectural patterns and state management strategies.

References

L&T Edutech